CS 320 Project Two: Summary and Reflections Report

Summary

Testing Approach and Effectiveness

For Project One, I used JUnit to test the Contact, Task, and Appointment services for Grand Strand Systems. My approach included unit testing, positive testing, negative testing, and boundary-value testing. I checked whether valid information was accepted, invalid information was rejected, and service operations behaved as expected. These tests helped me evaluate whether the code followed the requirements.

For the Contact feature, I tested contact IDs, names, phone numbers, and addresses. In ContactTest.java, the testPhoneMustBe10Digits() method checks phone numbers containing nine and eleven digits to verify the ten-digit requirement. In ContactServiceTest.java, I tested adding contacts, preventing duplicate IDs, deleting contacts, and updating information. However, some service tests could be stronger because checking that an operation does not throw an exception does not necessarily prove that the correct information was saved.

For the Task feature, I tested restrictions on task IDs, names, and descriptions. The tests check maximum lengths and reject null values. In TaskServiceTest.java, I tested adding, retrieving, deleting, and updating tasks. For example, assertThrows() checks that adding a task with a

duplicate ID produces an IllegalArgumentException. The update tests also verify that the task's name and description match the expected values. These assertions provide evidence that the service behaves according to its requirements.

For the Appointment feature, I tested appointment creation, invalid IDs, null dates, past dates, and descriptions that exceed the character limit. In AppointmentServiceTest.java, I tested adding appointments, preventing duplicate IDs, deleting appointments, and attempting to delete an appointment that does not exist. Additional tests for invalid setter inputs and boundary cases would help identify problems that the current tests might miss.

Eclipse reported **84.0% overall instruction coverage**, with 1,296 instructions covered and 246 missed out of 1,542. The reported package coverage was 87.3% for Appointment, 85.8% for Task, and 80.6% for Contact. Contact had the lowest package coverage and 126 missed instructions. These results indicate that the tests exercised much of the code, but coverage alone does not prove that every requirement works correctly. A test may execute a method without checking whether its result is correct. Therefore, I would strengthen the Contact update tests, verify the resulting values directly, and add tests for missing records and invalid updates.

My tests were organized around individual behaviors, which made failures easier to identify. I used assertEquals() to compare actual values with expected values and assertThrows() to verify expected exceptions. Keeping tests focused on specific behaviors also makes it easier to maintain the suite as the code changes. Overall, the results show that the tests provided useful coverage, while also identifying areas that need further testing.

Reflection

Testing Techniques and Their Practical Uses

The main techniques I used were unit testing, positive testing, negative testing, and boundary-value testing. Unit testing checks individual classes or methods. Positive testing verifies that valid inputs produce the expected results, while negative testing checks how the application responds to invalid inputs. Boundary-value testing examines values near or beyond permitted limits, such as a phone number that contains too few digits or a description that exceeds its maximum length.

These techniques are useful for applications that accept and process user information because they help identify defects early. However, I did not perform full integration, performance, security, or user-acceptance testing. Integration testing would help verify that connected components work together correctly. Performance testing would examine how the application behaves under heavy workloads, while security testing would look for weaknesses that could expose information. User-acceptance testing would help determine whether the application meets users' needs. These techniques would be important when preparing a complete application for release because passing unit tests does not guarantee that the entire system works correctly.

Caution, Complexity, and Bias

I learned that developers must be cautious because code can appear correct while failing in less common situations. Different methods may also handle similar situations differently. For example, TaskService.getTask() returns null when a task cannot be found, while

TaskService.updateTask() throws an exception when the task does not exist. Tests must account for these differences instead of assuming every method behaves the same way.

The project also demonstrated how classes depend on one another. The service methods use model objects, and the model setters enforce validation rules. If an update passes invalid information to a setter, the operation may fail even if the service method itself is written correctly. Testing these interactions carefully is important because a problem in one class can affect the behavior of another.

Testing my own code can introduce bias because I know what I intended the code to do. I might concentrate on normal situations and overlook unexpected inputs. To reduce this bias, I tested null values, duplicate IDs, invalid phone numbers, and excessive field lengths. In future projects, I would use a requirements checklist, review boundary cases systematically, and ask another developer to review important tests when possible. Independent review can help identify assumptions or gaps that I might overlook.

Discipline and Avoiding Technical Debt

Discipline is important because skipping tests or using weak assertions can allow defects to remain in the code. These problems may become more expensive to fix as other components begin to depend on the same behavior. Technical debt can also increase when developers repeatedly take shortcuts instead of addressing weaknesses properly.

To avoid technical debt, I would write tests alongside new functionality, run the test suite after making changes, and add regression tests when defects are discovered. For this project, I would improve the Contact update tests by checking the actual values after each operation. I would also

add tests for invalid Task updates, missing records, and invalid Appointment setter inputs. Reviewing coverage would help identify untested code, while meaningful assertions would help confirm that the code behaves correctly.

Overall, this project taught me that software quality requires more than code that compiles or passes a few tests. Developers must understand requirements, consider unexpected situations, examine how components interact, and improve weaknesses found during testing. Following these practices helps produce software that is more reliable and easier to maintain.